

RedNorte

Persistencia de Datos — RedNorte

Tecnología

- **Motor:** PostgreSQL 16 (imagen postgres:16-alpine)
- **Cliente:** [postgres](#) v3 — cliente SQL directo para Node.js/Bun (sin ORM)
- **Migraciones:** SQL crudo ejecutado desde archivos .sql con tabla de control `_migrations_history`

Arquitectura de Bases de Datos

Cada microservicio tiene su propia base de datos PostgreSQL independiente:

Microservicio	Base de Datos	Puerto Externo	Propósito
ms-users	users_db	5432	Usuarios, roles, autenticación
ms-waitlist	waitlist_db	5433	Lista de espera médica
ms-notifications	notifications_db	5434	Notificaciones persistentes

Esquemas

users_db (ms-users/migrations/0001_initial_schema.sql)

```
-- Roles del sistema
CREATE TABLE roles (
  id TEXT PRIMARY KEY,          -- ej: 'rol_patient', 'rol_doctor', 'rol_admin'
  key TEXT UNIQUE NOT NULL,     -- ej: 'patient', 'doctor', 'admin'
  description TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Usuarios registrados
CREATE TABLE users (
  id TEXT PRIMARY KEY,
  first_name TEXT NOT NULL,
  last_name TEXT,
  is_verified BOOLEAN DEFAULT FALSE,
```

```

    email TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,          -- hash Argon2
    role_id TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE RESTRICT
);

CREATE INDEX idx_users_email ON users(email);

```

waitlist_db (ms-waitlist/migrations/0001_initial_waitlist_schema.sql)

```

CREATE TABLE waitlist_entries (
    id SERIAL PRIMARY KEY,
    user_id TEXT NOT NULL,
    priority INTEGER NOT NULL DEFAULT 1, -- 1-4: Baja, Media, Alta, Urgencia
    status TEXT NOT NULL DEFAULT 'waiting', -- waiting, attending, finished,
    cancelled
    reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_waitlist_status ON waitlist_entries(status);

```

notifications_db (ms-notifications/migrations/0001_initial_notifications_schema.sql)

```

CREATE TABLE notifications (
    id SERIAL PRIMARY KEY,
    user_id TEXT NOT NULL,
    type VARCHAR(50) NOT NULL,
    message TEXT NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Migraciones

Cada microservicio ejecuta migraciones al iniciar mediante `bun run src/db/migrate.ts`:

1. Crea la tabla `_migrations_history` si no existe
2. Lee los archivos `.sql` ordenados desde `migrations/`
3. Ejecuta solo los archivos no registrados en el historial

4. Registra cada migración ejecutada para evitar duplicados

Conexión

Configuración vía variable de entorno DATABASE_URL:

```
postgres://usuario:password@host:5432/nombre_db
```

Pool de conexiones: máximo 10 conexiones, timeout de inactividad 20s, timeout de conexión 10s.

Seguridad y Respaldo

- Las contraseñas se almacenan con hash Argon2 (via Bun.password)
- Cada base de datos es independiente (aislamiento entre servicios)
- Volúmenes Docker persistentes: pgdata_users, pgdata_waitlist, notifications_data
- Los datos persisten entre reinicios de contenedores